

HMT – N32 (Cierre de Referee): ILP de K_{R12} , congelación por hash, y separación Definición vs Verificación

(Entrega unificada: `ventana_HMT_N32_referee_closeout.tex` + `hmt_n32_referee_closeout.py`)

Objetivo (lista cerrada)

Esta ventana cierra *exactamente* tres puntos que un referee suele exigir para no dejar huecos epistemológicos:

1. **Certificado ILP de K_{R12} :** fijar U_{12} , publicar el ILP exacto, y demostrar unicidad (sin usar α).
2. **Congelar semillas/rutas por hash:** listar explícitamente las “semillas” y la ruta D^{108} del rotor, de modo que cualquiera reejecute sin mencionar α .
3. **Separar definición vs verificación:** el texto debe distinguir qué es el motor/definición y qué es una comprobación numérica.

No hay más puntos escondidos en esta entrega.

Estado PASS/NO-PASS

Ítem	Descripción	Estado
R1	ILP $U \rightarrow K_{R12}$, U_{12} fijado, unicidad	PASS
R2	Semillas + ruta D^{108} congeladas con SHA256	PASS
R3	Separación Definición vs Verificación (sin circularidad)	PASS

1 R1. Certificado ILP de K_{R12} (sin dial)

1.1 Dato fijo: U_{12}

En el lenguaje del paquete 3.0-alfa, el extractor produce un vector $U_{12} \in \mathbb{Z}^{12}$, que se organiza por clases módulo 3 como tres bloques de 4 entradas:

$$U^{(1)} = (U_1, U_4, U_7, U_{10}) = (2378, -452, -668, -322), \quad (1)$$

$$U^{(2)} = (U_2, U_5, U_8, U_{11}) = (1406, 998, -204, -28), \quad (2)$$

$$U^{(3)} = (U_3, U_6, U_9, U_{12}) = (2479, -551, -371, -997). \quad (3)$$

Equivalentemente, el orden natural $U_1, \dots, U_{12}$ queda fijado por:

$$(U_1, U_2, U_3, U_4, U_5, U_6, U_7, U_8, U_9, U_{10}, U_{11}, U_{12}) = (2378, 1406, 2479, -452, 998, -551, -668, -204, -371, -322, -28, -997)$$

1.2 ILP explícito

El ILP que define K_{R12} usa variables enteras

$$K_1, \dots, K_{12} \in \{0, 1, \dots, 999\}.$$

Se imponen 12 ecuaciones lineales, bloque a bloque, usando la matriz de Hadamard H_4 :

$$H_4 = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & 1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix}.$$

Con la convención de intercalado

$$K^{(1)} = (K_1, K_4, K_7, K_{10}), \quad K^{(2)} = (K_2, K_5, K_8, K_{11}), \quad K^{(3)} = (K_3, K_6, K_9, K_{12}),$$

se exige

$$U^{(j)} = H_4 K^{(j)} \quad (j = 1, 2, 3).$$

Objetivo: *minimizar* 0 (es un ILP de factibilidad). El modelo en formato `.lp` se exporta como `HMT_N32_ILP_model.lp` por el script.

1.3 Unicidad (prueba corta, no computacional)

Como $\det(H_4) = 16 \neq 0$, el sistema $U^{(j)} = H_4 K^{(j)}$ tiene a lo sumo una solución racional

$$K^{(j)} = \frac{1}{4} H_4 U^{(j)}.$$

Por tanto, el sistema completo (12 ecuaciones) tiene a lo sumo una solución racional. Si además la solución resulta entera y cae en el cubo $[0, 999]^{12}$, entonces es la *única* solución entera admisible del ILP.

1.4 Solución (única) obtenida

El cálculo da

$$K^{(1)} = (234, 729, 621, 794), \tag{4}$$

$$K^{(2)} = (543, 659, 58, 146), \tag{5}$$

$$K^{(3)} = (140, 824, 914, 601), \tag{6}$$

por tanto

$$K_{R12}^{\text{CAN}} = (K_1, \dots, K_{12}) = (234, 543, 140, 729, 659, 824, 621, 058, 914, 794, 146, 601).$$

Importante: esta definición de K_{R12} no usa α en ningún punto.

2 R2. Congelación de semillas/rutas (por hash)

2.1 Semillas mínimas publicables

En los materiales internos aparece una parametrización mínima (sin knobs continuos) para las tres ejecuciones π, e, φ en base-3 (palabras w_6) y su permutación asociada:

Constante	w_6	permutación P sobre $\{0, 1, 2, 3, 4, 5\}$
π	010211	$[4, 0, 5, 3, 1, 2]$
e	121002	$[5, 1, 0, 2, 4, 3]$
φ	201220	$[2, 3, 5, 0, 4, 1]$

(Se congela también el bloque base $U_6 = (541, 478, 694, 923, 810, 870)$ que actúa como “normalización” del catálogo.)

2.2 Ruta canónica D^{108} del rotor

La ruta discreta se fija por la regla (3.0-alfa):

$$\beta_{(b)} \equiv 4b \pmod{12}, \quad b \in \{0, 1, \dots, 8\}.$$

Esto induce una ruta de longitud 108 sobre índices 1..12:

$$\text{idx}(k) = 1 + ((k - 1 \bmod 12) + 4 \lfloor (k - 1)/12 \rfloor) \bmod 12, \quad k = 1, \dots, 108.$$

El script exporta la lista completa y su SHA256 dentro de `HMT_N32_certificate.json`.

2.3 Ejecuciones π, e, φ (12 triadas base-1000)

Para evitar ambigüedad de librerías/precisión, se congela (y hashea) la ejecución como las primeras 12 triadas del desarrollo decimal fraccionario:

$$\pi \mapsto (141, 592, 653, 589, 793, 238, 462, 643, 383, 279, 502, 884), \quad (7)$$

$$e \mapsto (718, 281, 828, 459, 045, 235, 360, 287, 471, 352, 662, 497), \quad (8)$$

$$\varphi \mapsto (618, 033, 988, 749, 894, 848, 204, 586, 834, 365, 638, 117). \quad (9)$$

Estas listas (como strings) y su SHA256 se incluyen en el certificado.

3 R3. Definición vs Verificación

3.1 Definición (no usa α)

La parte *definicional* es exactamente:

$$\text{motor/ledger} \longrightarrow U_{12} \longrightarrow \text{ILP (factibilidad)} \longrightarrow K_{R12}^{\text{CAN}}.$$

Esto fija K_{R12} sin knobs y sin comparar con α .

3.2 Verificación (sí compara con α)

Una vez fijado K_{R12}^{CAN} , se define el lector de acarreo derecha→izquierda (base 1000) sobre 12 triadas:

$$S_i = \pi_i + e_i - \varphi_i - K_i + c_{i+1}, \quad c_i = \left\lfloor \frac{S_i}{1000} \right\rfloor, \quad A_i = S_i - 1000c_i.$$

El resultado (12 triadas) es

$$A = (7, 297, 352, 569, 283, 800, 997, 285, 105, 472, 380, 663),$$

que se interpreta como

$$\alpha \approx 0.007297352569283800997285105472380663, \quad \alpha^{-1} \approx 137.035999084 \dots$$

Crucial: esta comparación aparece sólo aquí, como *verificación*; no entra en la construcción de U_{12} ni del ILP.

Reproducibilidad

Ejecuta:

```
python3 hmt_n32_referee_closeout.py
```

Genera:

- HMT_N32_ILP_model.lp (ILP exacto),
- HMT_N32_certificate.json (hashes, semillas, ruta, U_{12} , K_{R12} , verificación α),
- HMT_N32_MANIFEST.sha256 (hashes de todos los outputs).